

```
In [4]: import pandas as pd  
import numpy as np
```

```
In [5]: df=pd.read_csv('ecommercepurchases.csv')  
df
```

Out[5]:

	Address	Lot	AM or PM	Browser Info	Company	Credit Card	E Dc
0	16629 Pace Camp Apt. 448\nAlexisborough, NE 77...	46 in	PM	Opera/9.56.(X11; Linux x86_64; sl-SI) Presto/2...	Martinez- Herman	6011929061123406	02/
1	9374 Jasmine Spurs Suite 508\nSouth John, TN 8...	28 rn	PM	Opera/8.93. (Windows 98; Win 9x 4.90; en-US) Pr...	Fletcher, Richards and Whitaker	3337758169645356	11/
2	Unit 0065 Box 5052\nDPO AP 27450	94 vE	PM	Mozilla/5.0 (compatible; MSIE 9.0; Windows NT ...	Simpson, Williams and Pham	675957666125	08/
3	7780 Julia Fords\nNew Stacy, WA 45798	36 vm	PM	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_0 ...	Williams, Marshall and Buchanan	6011578504430710	02/
4	23012 Munoz Drive Suite 337\nNew Cynthia, TX 5...	20 IE	AM	Opera/9.58.(X11; Linux x86_64; it-IT) Presto/2...	Brown, Watson and Andrews	6011456623207998	10/
...	...	...	...	...	...	...	...
9995	966 Castaneda Locks\nWest Juliafurt, CO 96415	92 XI	PM	Mozilla/5.0 (Windows NT 5.1) AppleWebKit/5352 ...	Randall- Sloan	342945015358701	03/
9996	832 Curtis Dam Suite 785\nNorth Edwardburgh, T...	41 JY	AM	Mozilla/5.0 (compatible; MSIE 9.0; Windows NT ...	Hale, Collins and Wilson	210033169205009	07/
9997	Unit 4434 Box 6343\nDPO AE 28026- 0283	74 Zh	AM	Mozilla/5.0 (Macintosh; U; Intel Mac OS X 10_7...	Anderson Ltd	6011539787356311	05/
9998	0096 English Rest\nRoystad, IA 12457	74 cL	PM	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_8;...	Cook Inc	180003348082930	11/
9999	40674 Barrett Stravenue\nGrimesville, WI 79682	64 Hr	AM	Mozilla/5.0 (X11; Linux i686; rv:1.9.5.20) Gec...	Greene Inc	4139972901927273	02/

10000 rows × 14 columns



```
In [14]: # 1. Display Top 10 Rows of the Dataset
df.head(10)
```

Out[14]:

	Address	Lot	AM or PM	Browser Info	Company	Credit Card	
0	16629 Pace Camp Apt. 448\nAlexisborough, NE 77...	46 in	PM	Opera/9.56.(X11; Linux x86_64; sl-SI) Presto/2...	Martinez- Herman	6011929061123406	C
1	9374 Jasmine Spurs Suite 508\nSouth John, TN 8...	28 rn	PM	Opera/8.93. (Windows 98; Win 9x 4.90; en-US) Pr...	Fletcher, Richards and Whitaker	3337758169645356	1
2	Unit 0065 Box 5052\nDPO AP 27450	94 vE	PM	Mozilla/5.0 (compatible; MSIE 9.0; Windows NT ...	Simpson, Williams and Pham	675957666125	C
3	7780 Julia Fords\nNew Stacy, WA 45798	36 vm	PM	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_0 ...	Williams, Marshall and Buchanan	6011578504430710	C
4	23012 Munoz Drive Suite 337\nNew Cynthia, TX 5...	20 IE	AM	Opera/9.58.(X11; Linux x86_64; it-IT) Presto/2...	Brown, Watson and Andrews	6011456623207998	1
5	7502 Powell Mission Apt. 768\nTravisland, VA 3...	21 XT	PM	Mozilla/5.0 (Macintosh; U; PPC Mac OS X 10_8_5...	Silva- Anderson	30246185196287	C
6	93971 Conway Causeway\nAndersonburgh, AZ 75107	96 Xt	AM	Mozilla/5.0 (compatible; MSIE 7.0; Windows NT ...	Gibson and Sons	6011398782655569	C
7	260 Rachel Plains Suite 366\nCastroberg, WV 24...	96 pG	PM	Mozilla/5.0 (X11; Linux i686) AppleWebKit/5350...	Marshall- Collins	561252141909	C
8	2129 Dylan Burg\nNew Michelle, ME 28650	45 JN	PM	Mozilla/5.0 (Macintosh; U; Intel Mac OS X 10_7...	Galloway and Sons	180041795790001	C
9	3795 Dawson Extensions\nLake Tinafort, ID 88739	15 Ug	AM	Mozilla/5.0 (X11; Linux i686; rv:1.9.7.20) Gec...	Rivera, Buchanan and Ramirez	4396283918371	C

```
In [15]: # 2. Check Last 10 Rows of The Dataset
df.tail(10)
```

Out[15]:

	Address	Lot	AM or PM	Browser Info	Company	Credit Card	E Dc
9990	75731 Molly Springs\nWest Danielle, VT 96934-5102	93 ty	PM	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_4;...	Pace, Vazquez and Richards	869968197049750	04/
9991	PSC 8165, Box 8498\nAPO AP 60327-0346	50 dA	AM	Mozilla/5.0 (compatible; MSIE 8.0; Windows NT ...	Snyder Inc	4221582137197481	02/
9992	885 Allen Mountains Apt. 230\nWallhaven, LA 16995	40 vH	PM	Mozilla/5.0 (Macintosh; PPC Mac OS X 10_6_5) A...	Wells Ltd	4664825258997302	10/
9993	7555 Larson Locks Suite 229\nEllisburgh, MA 34...	72 jg	PM	Mozilla/5.0 (Macintosh; U; Intel Mac OS X 10_8...	Colon and Sons	30025560104631	10/
9994	6276 Rojas Hollow\nLake Louis, WY 56410-7837	93 Ex	PM	Opera/9.68.(X11; Linux x86_64; sl-SI) Presto/2...	Ritter-Smith	3112186784121077	01/
9995	966 Castaneda Locks\nWest Juliafurt, CO 96415	92 XI	PM	Mozilla/5.0 (Windows NT 5.1) AppleWebKit/5352 ...	Randall-Sloan	342945015358701	03/
9996	832 Curtis Dam Suite 785\nNorth Edwardburgh, T...	41 JY	AM	Mozilla/5.0 (compatible; MSIE 9.0; Windows NT ...	Hale, Collins and Wilson	210033169205009	07/
9997	Unit 4434 Box 6343\nDPO AE 28026-0283	74 Zh	AM	Mozilla/5.0 (Macintosh; U; Intel Mac OS X 10_7...	Anderson Ltd	6011539787356311	05/
9998	0096 English Rest\nRoystad, IA 12457	74 cL	PM	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_8;...	Cook Inc	180003348082930	11/
9999	40674 Barrett Stravenue\nGrimesville, WI 79682	64 Hr	AM	Mozilla/5.0 (X11; Linux i686; rv:1.9.5.20) Gec...	Greene Inc	4139972901927273	02/



In [19]: # 3. Check Datatype of each column
df.dtypes

```
Out[19]: Address      object
        Lot          object
        AM or PM      object
        Browser Info  object
        Company       object
        Credit Card   int64
        CC Exp Date   object
        CC Security Code int64
        CC Provider   object
        Email         object
        Job           object
        IP Address    object
        Language      object
        Purchase Price float64
        dtype: object
```

```
In [6]: # 4. Check null Values in the DataSet
        df.isnull().sum()
```

```
Out[6]: Address      0
        Lot          0
        AM or PM      0
        Browser Info  0
        Company       0
        Credit Card   0
        CC Exp Date   0
        CC Security Code 0
        CC Provider   0
        Email         0
        Job           0
        IP Address    0
        Language      0
        Purchase Price 0
        dtype: int64
```

```
In [23]: # 5. How many rows & columns are there in Dataset ?
        print("number of rows=",df.shape[0])
        print("number of coulumn=",df.shape[1])
```

```
number of rows= 10000
number of coulumn= 14
```

```
In [29]: # 6. Highest & Lowest Purchase Prices
        print("high price",df['Purchase Price'].max())
        print("low price",df['Purchase Price'].min())
```

```
high price 99.99
low price 0.0
```

```
In [32]: # 7. Average Purchase Price
        df['Purchase Price'].mean()
```

```
Out[32]: np.float64(50.347302)
```

```
In [13]: # 8. How Many People Have their French 'fr' as their Language ?
        print("number of people have their french 'fr' as their language=",len(df[df['Langu
```

number of people have their french 'fr' as their language= 1097

```
In [17]: #9 job titles contains Engineer----->
eng_jobs=df[df['Job'].str.contains('Engineer')].shape[0]
print("number of job titles contains 'engineer'=",eng_jobs)
```

number of job titles contains 'engineer'= 453

```
In [18]: # 10. Find Email of the person with the following IP address: 132.207.160.22
df[df['IP Address']=='132.207.160.22']['Email']
```

```
Out[18]: 2    amymiller@morales-harrison.com
Name: Email, dtype: object
```

```
In [20]: # 11. How many people have mastercard as their credit card provider & made a purcha
count=df[(df['CC Provider']=='Mastercard') & (df['Purchase Price']>50)].shape[0]
```

Number of people: 405

```
In [21]: # 12. Find Email of the person with the following credit card number: 4664825258997
df[df['Credit Card']==4664825258997302]['Email']
```

```
Out[21]: 9992    bberry@wright.net
Name: Email, dtype: object
```

```
In [22]: # 13. How many people purchase during the AM & how many people purchase during the
df['AM or PM'].value_counts()
```

```
Out[22]: AM or PM
PM      5068
AM      4932
Name: count, dtype: int64
```

```
In [67]: #14 How many people have a credit card that expires in 2020 ?----->
df[df['CC Exp Date']=='20'].count()
```

```
Out[67]: Address      0
Lot      0
AM or PM    0
Browser Info  0
Company     0
Credit Card  0
CC Exp Date  0
CC Security Code  0
CC Provider  0
Email      0
Job      0
IP Address  0
Language   0
Purchase Price  0
dtype: int64
```

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: